

Continuum Simulator: monitoraggio ambientale e aggregazione distribuita Edge–Cloud

Alessandro Vassallo – Matricola 0366572
Università degli Studi di Roma Tor Vergata
Sistemi Distribuiti e Cloud Computing – A.A. 2025/26

Sommario—Il monitoraggio di sensori distribuiti richiede di combinare l’elaborazione locale delle osservazioni con una vista complessiva delle zone monitorate. Questo lavoro presenta un’applicazione in Go che riproduce misure ambientali *Sensor.Community* e distribuisce la computazione tra Edge e Cloud. Tredici gateway validano temperatura, umidità e pressione e producono statistiche su finestre di cinque minuti. Nel Cloud, worker appartenenti allo stesso consumer group Kafka aggregano i dati su finestre di quindici minuti; un componente finale combina i contributi delle partizioni. L’identità degli aggregati è indipendente dal numero di worker, mentre watermark espliciti prodotti dagli Edge e marker terminali consentono di distinguere contributi assenti da contributi ancora attesi. La relazione descrive la costruzione del workload, le scelte di comunicazione, il deployment automatizzato e le verifiche disponibili. Le prove AWS preliminari conservano i conteggi degli eventi, ma non superano i controlli di orchestrazione; non vengono quindi utilizzate per affermare la scalabilità. Sono definiti il metodo per la successiva valutazione e i limiti relativi a perdite MQTT, stato volatile e ripristino dopo un guasto.

Index Terms—compute continuum, Edge–Cloud, Go, Kafka, aggregazione, scalabilità

I. INTRODUZIONE E OBIETTIVI

Le osservazioni ambientali generate in zone differenti possono essere elaborate vicino alle sorgenti per descrivere la situazione locale e successivamente combinate per ottenere una vista dell’intero insieme monitorato. Trasferire al Cloud ogni singolo campione concentra in un unico livello sia la validazione sia il lavoro di aggregazione. Quando l’analisi richiede statistiche componibili, una prima elaborazione all’Edge permette invece di inviare riepiloghi temporali, conservando le informazioni necessarie al calcolo globale.

Continuum Simulator realizza questa separazione mediante una pipeline di servizi Go. Il caso di studio riguarda misure di temperatura, umidità e pressione ottenute da sensori BME280. L’analisi locale produce statistiche per zona; il risultato Cloud riassume le osservazioni di tutte le zone su un intervallo temporale comune. La media globale è riferita ai campioni validi ricevuti: non è una stima della temperatura media geografica europea, che richiederebbe un diverso modello di pesatura e di rappresentatività spaziale.

Il progetto segue la traccia A2 [1]. Essendo svolto individualmente, seleziona il requisito di *scalabilità orizzontale* fra le due alternative previste, scalabilità e tolleranza ai guasti.

Bozza di lavoro dell’11 settembre 2026. La campagna sperimentale di scalabilità e la valutazione delle condizioni di rete sono da completare.

La domanda sperimentale è se l’aggiunta di repliche Cloud permetta di sostenere un maggiore carico senza degradare i tempi di disponibilità dei risultati. Il placement dei componenti è deciso a progetto e il numero di worker viene fissato prima di ogni esecuzione; non è implementato autoscaling durante il replay.

Le principali scelte sviluppate nel progetto sono una topologia derivata dai dati, l’aggregazione gerarchica di statistiche componibili e un protocollo di finalizzazione basato su watermark espliciti emessi dalle sorgenti Edge. La parallelizzazione Cloud conserva sei partizioni logiche anche quando cambia il numero degli esecutori. In questo modo i confronti fra configurazioni mantengono invariata la computazione richiesta e il significato del risultato.

II. REQUISITI E PERIMETRO

La traccia richiede almeno i livelli Edge e Cloud, l’uso di Go, comunicazione persistente fra componenti, deployment automatizzato e configurabile, esecuzione con Docker Compose su EC2 e valutazione delle comunicazioni sotto condizioni di rete emulate [1]. Il progetto adotta processi containerizzati, MQTT per l’ingresso delle misure e Kafka nei due passaggi Cloud. La persistenza richiesta è affidata a Kafka; la telemetria MQTT usa una semantica che ammette perdite, discusse nella Sez. VI.

Non viene introdotto un livello Fog. Le tre grandezze considerate sono riassumibili con conteggi, somme ed estremi: nella soluzione corrente non emerge un’ulteriore elaborazione intermedia con una responsabilità distinta. La scelta mantiene esplicite l’analisi locale e la sintesi globale e riduce i componenti da configurare e misurare. Le zone Edge sono domini logici: più zone possono condividere un host, con limiti di CPU e memoria per container. Le coordinate dei sensori definiscono i gruppi di dati, non la posizione fisica delle macchine EC2 né il ritardo della rete.

A. Dataset e costruzione dello scenario sperimentale

Lo scenario sperimentale è stato costruito a partire dal dataset pubblico *Sensor.Community* relativo ai sensori ambientali BME280. È stata considerata la finestra temporale di gennaio 2025 e il dataset, di dimensione pari a circa 6.36 GB, è stato analizzato in modo incrementale per evitare il caricamento completo in memoria.

Una prima fase di preprocessing ha permesso di verificare la qualità dei dati, filtrare le osservazioni esterne alla finestra temporale e individuare i sensori dotati di coordinate geografiche valide. Delle variabili disponibili, le coordinate geografiche sono state utilizzate per la costruzione della topologia, mentre temperatura, umidità e pressione sono state mantenute come misure da riprodurre durante l'esecuzione. Le variabili `altitude` e `pressure_sealevel`, completamente prive di valori, sono state escluse.

Lo studio è stato limitato all'area europea. Per ogni sensore è stata determinata una posizione rappresentativa e le coordinate geografiche WGS84 sono state proiettate nel sistema metrico EPSG:3035, in modo da applicare il clustering su distanze spaziali coerenti. La popolazione risultante comprende 5102 sensori europei localizzabili.

La suddivisione in Edge Site è stata ottenuta tramite K-Means sulle coordinate proiettate. Sono stati valutati valori di k compresi tra 5 e 15 utilizzando il Silhouette Score come criterio principale di selezione, affiancato dall'inertial e dal raggio massimo dei cluster come indicatori diagnostici. La configurazione selezionata è risultata $k = 13$, con Silhouette Score pari a circa 0.521; l'elbow method individua invece $k = 9$, utilizzato solamente come riferimento diagnostico.



Figura 1. Topologia geografica della popolazione europea e centroidi dei 13 Edge Site.

Una volta determinata la topologia sull'intera popolazione europea, è stato costruito il workload sperimentale selezionando 150 sensori. I sensori sono stati distribuiti nel modo più uniforme possibile tra i 13 Edge Site (11–12 sensori per Edge). In questo modo ogni Edge partecipa in maniera comparabile agli esperimenti, pur mantenendo le differenze naturali nel rate di generazione degli eventi.

Infine, la fase di preprocessing produce due tipi di artefatti utilizzati durante l'esecuzione: il file `kmeans_topology.csv`, che associa i sensori selezionati agli Edge Site, e 13 shard di replay, uno per ciascun Edge, contenenti identificativo del sensore, timestamp, pressione, temperatura e umidità.

III. ARCHITETTURA E SCELTE PROGETTUALI

A. Flusso dei dati e responsabilità

La Fig. 2 rappresenta la pipeline. Ciascun sito comprende un broker Mosquitto e un Edge Gateway; il Simulator corrispondente pubblica JSON sul topic MQTT `sensors/<sensorID>/telemetry`. Il gateway valida le misure e genera un `EdgeAggregate` per zona e finestra non vuota di cinque minuti. Il record contiene eventi, conteggi validi e non validi, somma, media, minimo e massimo per ciascuna grandezza.

Kafka disaccoppia la produzione Edge dal consumo Cloud e mantiene i record in un log persistente [3]. Il topic `edge-aggregates` ha sei partizioni. La chiave `edge_id` e il partizionatore hash mantengono `EdgeAggregate`, `EdgeWatermark` e `marker` terminale dello stesso Edge nella medesima partizione e nello stesso ordine. Il numero di partizioni e la membership degli Edge rimangono invariati per l'intera run.

I Cloud Worker condividono un consumer group: ciascuna partizione è assegnata a un solo membro del gruppo. La topologia Edge è proiettata sulle partizioni con lo stesso hash del producer; ogni worker mantiene uno stato separato per ciascuna partizione assegnata, indipendente dalla propria identità.

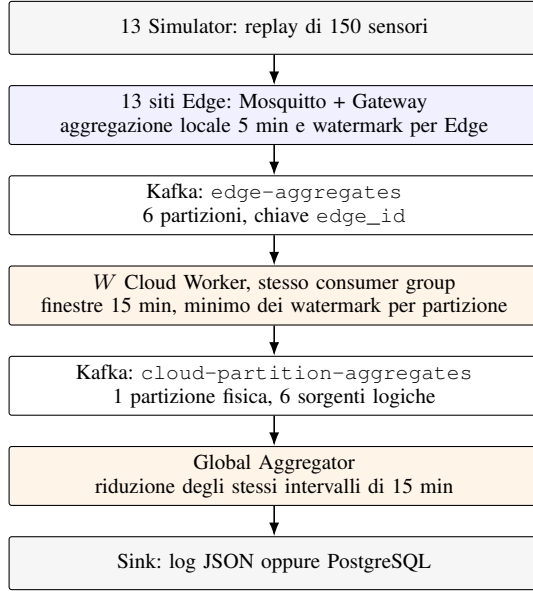


Figura 2. Architettura logica. I due topic risiedono sullo stesso broker Kafka. Le repliche dei worker non modificano le sei sorgenti logiche del Global.

I contributi Edge sono combinati su finestre di quindici minuti. Gli output sono `CloudPartitionAggregate`, identificati da partizione sorgente e confini temporali, pubblicati sul secondo topic Kafka. Il Global Aggregator combina i partial con la stessa coppia di confini e produce il risultato nel sink.

B. Aggregazione componibile

Per una metrica m , ogni aggregato conserva il numero di misure valide n_m , il numero di misure non valide u_m e la somma s_m , oltre agli estremi. La combinazione di contributi disgiunti j applica

$$n_m = \sum_j n_{m,j}, \quad u_m = \sum_j u_{m,j}, \quad s_m = \sum_j s_{m,j}. \quad (1)$$

La media si ottiene da s_m/n_m se $n_m > 0$; altrimenti media ed estremi sono nulli. Minimo e massimo si combinano considerando soltanto contributi validi. La somma dei conteggi validi e non validi deve coincidere con il numero di eventi dell'aggregato per ciascuna metrica.

Conservare la somma evita la media non pesata delle medie locali, che sarebbe errata in presenza di numerosità differenti. L'operazione è componibile in aritmetica esatta; con valori floating point l'equivalenza numerica va verificata con una tolleranza. I cinque minuti Edge dividono esattamente i quindici minuti Cloud, per cui ogni aggregato locale appartiene a una sola finestra Cloud. Un aggregato che attraversa un confine Cloud viene rifiutato.

C. Progresso, contributi vuoti e completamento

L'Edge mantiene una finestra corrente in event time. L'arrivo di un evento della finestra successiva determina la pubblicazione della precedente, seguita da `EdgeWatermark(e, t)`.

Tale controllo certifica che l'Edge e non pubblicherà successivamente aggregati con `window_end` minore o uguale a t . Il watermark è monotono. L'EOS ricevuto dal Simulator chiude l'ultima finestra; dopo averne completato la scrittura Kafka, l'Edge pubblica il distinto marker `EdgeEndOfInput`. Gli eventi appartenenti a finestre già chiuse sono scartati e conteggiati; il contratto richiede shard ordinati temporalmente.

Una partizione Cloud può contenere Edge con velocità differenti. Il worker memorizza, per ogni Edge atteso nella partizione P , il watermark W_e e il flag terminale. Finché un Edge attivo non ha prodotto un watermark, non è possibile certificarne il progresso. La frontiera locale è

$$W_P = \min_{e \in E_P, e \text{ attivo}} W_e. \quad (2)$$

Solo le finestre con fine non successiva a W_P sono definitive. Un Edge terminato non partecipa più al minimo; la cessazione temporanea del traffico, invece, non equivale alla conclusione della sorgente.

Il worker pubblica prima i partial definitivi e poi il controllo di progresso `PartitionProgress(P, t)`, con $t = W_P$. Quando tutti gli Edge di P terminano, pubblica in ordine gli ultimi partial e infine `PartitionEndOfReplay(P)`; una partizione senza Edge produce direttamente tale controllo. Un aggregato ricevuto dietro un watermark già certificato viola il protocollo e invalida la run: una finestra emessa non viene riaperta. L'ordine fra dati e controlli permette una rappresentazione sparsa: l'assenza di un partial, dopo il relativo progresso o EOS, equivale a un contributo zero.

È stata considerata anche un'alternativa in cui il Cloud conoscesse soltanto le partizioni Kafka, senza la membership degli Edge, e stimasse la frontiera come massimo event time osservato meno un ritardo. La soluzione ridurrebbe l'accoppiamento con la topologia, ma un Edge veloce potrebbe anticipare la chiusura rispetto a uno lento; inoltre il ritardo sarebbe un parametro euristico. Si è quindi preferita la certificazione esplicita degli Edge e il minimo per partizione, privilegiando la correttezza nel deployment statico adottato.

Il Global conosce soltanto le sei partizioni sorgente. Per una finestra attende da ciascuna un partial oppure un controllo di progresso/EOS che certifichi il contributo zero. Non introduce nuove finestre, non ricalcola watermark e non usa timeout per dichiarare completi risultati incompleti. Il replay termina dopo gli EOS di tutte le partizioni e l'emissione delle finestre pendenti.

D. Parallelismo indipendente dal risultato

Con $W = 1, 2, 4, 6$ cambia il numero di esecutori, non quello delle partizioni sorgente (Tab. I). Anche con un solo worker la pipeline passa dal secondo topic e dal Global. In presenza di dati in tutte le partizioni si producono sei partial per finestra; in caso contrario alcuni contributi sono certificati come zero. Il Global non aggrega per identità del worker.

Ogni worker usa letture concorrenti ma un solo ciclo di elaborazione; il parallelismo computazionale cresce quindi tramite processi aggiuntivi. Sei partizioni costituiscono il limite del parallelismo utile del gruppo. La distribuzione delle

Tabella I
DISTRIBUZIONE BILANCIATA DELLE SEI PARTIZIONI. L'ASSEGNAZIONE EFFETTIVA DEGLI IDENTIFICATIVI È DECISA DAL CONSUMER GROUP.

Worker	Partizioni per worker	Sorgenti logiche Global
1	6	6
2	3, 3	6
4	2, 2, 1, 1	6
6	1, 1, 1, 1, 1, 1	6

Tabella II
LIBRERIE GO DIRETTAMENTE UTILIZZATE DAL PROGETTO.

Libreria	Versione	Responsabilità
Paho MQTT	1.5.1	Client MQTT Simulator/Edge
kafka-go	0.4.51	Producer, consumer group e offset
goavro/v2	2.15.0	Codifica e decodifica Avro
pgx/v5	5.10.0	Sink PostgreSQL e pool connessioni
yaml.v3	3.0.1	Lettura configurazioni esperimenti

partizioni non garantisce un uguale costo: il numero di Edge e il volume dei loro aggregati possono differire. Il broker condiviso e il Global restano inoltre possibili colli di bottiglia.

IV. IMPLEMENTAZIONE E PIATTAFORMA SOFTWARE

A. Organizzazione e contratti

La directory `cmd` contiene i programmi Simulator, Edge, Cloud Worker e Global Aggregator. In `internal`, i package `model`, `cloudworker` e `globalaggregator` separano i contratti e l'elaborazione dall'accesso ai broker. La Tab. II riporta le dipendenze dirette dichiarate in `go.mod`; il modulo richiede Go 1.25.0.

La telemetria MQTT e il sink di log usano JSON. Gli aggregati, gli `EdgeWatermark` e i `PartitionProgress` sono record Avro binari con schema incorporato nei binari, senza Schema Registry o prefissi aggiuntivi; i marker terminali hanno payload vuoto e tipo nell'header Kafka. Avro descrive esplicitamente campi e tipi del record [4]; nel progetto producer e consumer devono condividere la stessa versione degli schemi. Le metriche nullable distinguono l'assenza di valori validi da un valore numerico nullo. I timestamp sono rappresentati come nanosecondi Unix e i contatori sono controllati prima della conversione da `uint64` a interi Avro con segno. Il decoder rifiuta record malformati e byte residui.

La validazione Edge ammette temperatura fra -40 e 85°C , umidità relativa fra 0 e 100%, pressione fra 30.000 e 110.000 Pa. Una misura mancante, non finita o fuori intervallo incrementa il relativo conteggio di invalidità senza necessariamente eliminare le altre misure dell'evento. Un evento con struttura o identificativi non validi è invece scartato e registrato nelle metriche operative.

B. Code, pubblicazione e deduplica

Simulator ed Edge impiegano code di ingresso limitate. Quando la coda è piena, la nuova telemetria è scartata e il contatore di drop viene incrementato. Nel Simulator il pacing continua indipendentemente dalla velocità di egress.

La telemetria MQTT usa QoS 0; l'EoS usa QoS 1 ed è accodato dopo i dati già accettati, attendendo il completamento della pubblicazione. All'Edge uno slot aggiuntivo è riservato all'EoS, che non può essere perso per saturazione della capacità destinata alla telemetria. La readiness del gateway diventa positiva soltanto dopo connessione e subscription ai topic richiesti.

I writer Kafka sono sincroni e richiedono gli acknowledgment del broker. Aggregati, watermark e marker terminale di ciascun Edge usano la stessa chiave e lo stesso writer: l'ultimo aggregato precede quindi `EdgeEndOfInput` nello stream della partizione. I worker accumulano, per ogni partizione, il massimo offset successivo elaborato e lo committano in batch configurabili, sempre dopo il successo di elaborazione e pubblicazione. Gli offset pendenti sono scaricati al termine regolare della generazione; il Global effettua il commit dopo elaborazione e scrittura nel sink.

Worker e Global riconoscono i duplicati nelle finestre pendenti e conservano anche l'ultimo record per sorgente. Un duplicato identico non modifica lo stato; un duplicato conflittuale o un nuovo contributo precedente al progresso già certificato causa errore. Non è mantenuto uno storico illimitato. Nel sink PostgreSQL l'identificativo dell'aggregato è univoco e l'inserimento usa `ON CONFLICT DO NOTHING`. Queste scelte non realizzano una transazione atomica fra stato applicativo, output e offset.

C. Deployment e riproducibilità

`cmd/deploygen` genera i file Compose a partire dalla topologia e dalla configurazione YAML, includendo numero di partizioni, repliche Cloud e membership Edge richiesta dai worker. L'infrastruttura Terraform prevede quattro host EC2: Simulator, Edge, Cloud Core e Workers. Cloud Core ospita Kafka e il Global; l'host Workers ospita le repliche da confrontare. Più siti Edge condividono quindi una macchina, mentre i confini Simulator-Edge e Edge-Cloud attraversano host distinti. Le configurazioni predefinite indicano istanze `t3.small` per tre ruoli e `t3.medium` per Cloud Core; una valutazione finale deve riportare i valori effettivi della campagna.

Il profilo pilota limita ogni worker a 0,25 CPU e 128 MiB, ogni gateway a 0,08 CPU e 64 MiB, Kafka a 1 CPU e 2.048 MiB e il Global a 0,50 CPU e 256 MiB. Le quote sono tetti di utilizzo, non core fisici riservati. Aumentare W aumenta il budget aggregato dei worker, ma la capacità dell'host rimane un vincolo. I template locali utilizzano Apache Kafka 4.3.0; il manifest delle immagini identifica il software effettivamente distribuito.

`deploy-pilot.sh` distribuisce esplicitamente configurazione e immagini; `run-full.sh` separa l'eventuale aggiornamento Terraform dall'esecuzione, mentre `run-experiment.sh` coordina readiness, stabilizzazione del gruppo, avvio comune, attesa degli EOS e raccolta degli artefatti. I risultati includono configurazione effettiva, log, statistiche dei container e offset Kafka. Un collector Go dedicato legge gli offset senza entrare nei consumer group; le

sue risorse restano separate da quelle del broker. I riepiloghi sono prodotti da script Python. Il notebook offline usa pandas, NumPy, scikit-learn, pyproj e Matplotlib; Bash, Docker Compose e Terraform supportano preparazione e deployment. Non è richiesto un framework di stream processing esterno: windowing e finalizzazione sono implementati nel codice Go.

V. VERIFICHE E VALUTAZIONE SPERIMENTALE

A. Verifiche di correttezza

Le suite Go verificano codec Avro, attesa degli Edge lenti, partizioni vuote, progresso, EOS, duplicati e conflitti. Controllano inoltre l'ordine pubblicazione-commit e l'equivalenza della computazione con $W = 1, 2, 4, 6$. Il confronto riguarda gli aggregati e le statistiche prodotti, non il tempo di esecuzione. La suite ordinaria `go test ./... -count=1`, rieseguita l'11 settembre 2026 durante la preparazione della bozza, è stata superata. I programmi Simulator ed Edge non hanno test Go propri nella versione ispezionata: la copertura automatizzata non va estesa per inferenza a tutta la pipeline.

Un test opt-in utilizza broker Kafka reali per esercitare consumer group e i due passaggi sul log; la verifica registrata nel documento di contratto del 9 settembre riporta otto casi superati, con tredici Edge oppure un solo Edge e cinque partizioni vuote. Tale prova non è stata rieseguita per redigere questa bozza e non sostituisce un benchmark completo MQTT-Edge-Cloud. Gli script di deployment includono inoltre test Python per parser, configurazioni, quote e controlli di qualità. Le integrazioni opzionali richiedono dipendenze esplicitamente abilitate; gli esiti e gli eventuali test saltati devono accompagnare la verifica finale.

B. Evidenze preliminari disponibili

Nel workspace sono conservate due prove AWS, datate 9 e 10 settembre 2026, entrambe con un worker e accelerazione $A = 10.000$. I riepiloghi riportano, in ciascuna prova, 2.106.784 eventi offerti, elaborati all'Edge e rappresentati nei risultati globali. Sono presenti 105.608 aggregati Edge e 2.710 finestre globali; i contatori di drop e di finestre globali incomplete sono nulli.

Tuttavia entrambe le esecuzioni hanno `status=failed` e un esito negativo dei controlli di qualità (`quality_status=fail`). Il motivo registrato è `orchestration_failed`. Nella prova del 10 settembre il log di orchestrazione segnala Kafka unhealthy nel controllo conclusivo. L'uguaglianza dei conteggi è un'evidenza diagnostica utile, ma non annulla il fallimento del protocollo di validazione. Queste prove non sono considerate run valide della campagna; non vengono usate per calcolare speedup o sostenere conclusioni sulla scalabilità.

Le due prove usano la strumentazione antecedente all'attuale collector Go del lag. Il costo della raccolta e la cadenza di campionamento rendono inappropriato un confronto diretto delle risorse del broker fra vecchie e nuove run. La campagna deve quindi essere ripetuta con release e strumentazione uniformi. Al momento della bozza non è disponibile una

serie AWS valida e ripetuta con tutti i valori di W né una valutazione con `tc-netem`.

C. Disegno della campagna da completare

Il primo passo è calibrare A in modo che Simulator ed Edge sostengano il carico senza perdite e senza accumulare un ritardo di generazione crescente. La baseline locale usa $A = 20.000$, i file di confronto Cloud $A = 40.000$ e il pilota AWS $A = 10.000$: sono parametri configurati, non capacità misurate. Il carico definitivo deve essere fissato dopo la calibrazione su EC2.

Il confronto principale prevede $W = 1, 2, 4$, con $W = 6$ come punto aggiuntivo. Per ogni valore si mantengono uguali dataset, finestre, code, immagini, quote per replica, numero di partizioni e strumentazione. Si eseguono più livelli di carico e almeno cinque ripetizioni per configurazione, alternando l'ordine dei valori di W . Si riportano mediana e dispersione, con intervalli di confidenza ove il numero di osservazioni lo consenta. Il replay finito comprende i transitori di avvio e svuotamento: essi devono essere dichiarati, ed eventualmente analizzati anche separatamente.

La prova a carico costante isola l'effetto delle repliche su tempo di completamento e backlog. Per dimostrare il requisito della traccia occorre anche aumentare il carico e verificare se più repliche mantengano comparabili i tempi di risposta. Il carico dei worker è il rate degli aggregati Edge, non soltanto quello degli eventi grezzi. A finestre fisse, aggiungere sensori a una zona già attiva può aumentare il lavoro Edge senza aumentare il numero di record Cloud. L'accelerazione del replay aumenta invece anche la frequenza di chiusura delle finestre rispetto al tempo reale.

La rete deve essere valutata introducendo separatamente ritardo e limiti di banda nei collegamenti Simulator-Edge ed Edge-Cloud/Kafka, poi combinandoli in scenari controllati. I valori dei profili, le interfacce e le direzioni interessate dovranno essere registrati negli artefatti. Questa estensione `tc-netem` non è ancora implementata; nessun dato di questa bozza ne presuppone l'esecuzione.

D. Metriche e criteri di accettazione

Si distinguono eventi offerti, accettati nelle code, ricevuti, elaborati e inclusi nei risultati. Una run del confronto senza perdite deve soddisfare $N_{\text{offerti}} = N_{\text{Edge}} = N_{\text{Global}}$, avere EOS completi, offset finali allineati, assenza di errori di protocollo, restart e OOM, e superare i controlli del runner. I confronti fra repliche devono verificare anche le statistiche per finestra, perché i soli conteggi non dimostrano l'equivalenza numerica dei risultati. Le run scartate vanno comunque censite.

Il throughput utile end-to-end è il numero di eventi rappresentati nei risultati diviso per la durata dichiarata della misura. Il throughput Cloud va espresso anche in aggregati Edge/s. Per un workload fisso, lo speedup può essere espresso come $S_W = T_1/T_W$ e l'efficienza come $E_W = S_W/W$, specificando che T_W include il pacing quando si usa la durata end-to-end. Una sorgente che impone il rate può rendere simili

i tempi di completamento anche se la capacità interna aumenta; per questo si osservano anche lag e ritardi.

Il lag Kafka è una differenza di offset, quindi un numero di record, non una latenza in secondi. I topic contengono anche controlli di progresso ed EOS. Il suo massimo è campionato e può sottostimare il picco; le query fallite o incomplete non devono essere sostituite con zero. CPU e RAM vanno riportate per ruolo e come totale delle repliche. Sugli host T3 occorre conservare anche le metriche dei crediti CPU e la modalità di funzionamento, per non attribuire al parallelismo effetti dovuti al throttling.

Per i tempi di disponibilità delle finestre, sottrarre il timestamp storico `window_end` da `emitted_at` produrrebbe un intervallo privo di significato prestazionale. Una metrica da introdurre nella campagna usa invece il fine-finestra riportato sulla timeline accelerata:

$$D(w) = T_{\text{emit}}(w) - \left(T_0 + \frac{t_{\text{end}}(w) - t_0}{A} \right). \quad (3)$$

Essa misura il ritardo rispetto alla chiusura nominale e include attesa delle sorgenti, code e servizio; non è la sola latenza di rete né la latenza di commit del sink. Richiede orologi sincronizzati. La finestra finale chiusa da EOS va analizzata separatamente, perché può terminare prima del suo confine nominale. Il collector corrente conserva i timestamp originali, ma non calcola questa metrica: i suoi percentili non sono ancora risultati disponibili.

VI. LIMITAZIONI E DISCUSSIONE

A. Persistenza e guasti

La persistenza del log Kafka non implica il ripristino dello stato delle finestre. Worker e Global mantengono gli aggregati in RAM; i Worker committano in batch soltanto offset già elaborati, ma non esiste un checkpoint coordinato fra offset e stato. Un crash può quindi perdere contributi già consumati e committati, che non verrebbero riletti al riavvio. Non è garantita una semantica exactly-once end-to-end. Il sink PostgreSQL finale non è uno state store degli aggregatori.

Per questo motivo un rebalance dopo l'inizio dell'elaborazione invalida la run; il worker rifiuta inoltre la ripartenza da offset già committati. Le repliche devono essere pronte prima del replay e restare stabili fino al termine. Un errore richiede una nuova esecuzione completa con topic e offset puliti. Il broker Kafka è singolo, con fattore di replica uno: gli acknowledgment richiesti non realizzano alta disponibilità. La tolleranza ai guasti non è il requisito individuale selezionato.

B. Completezza, carico e rappresentatività

QoS 0 e code limitate possono perdere telemetria. Il protocollo EOS certifica il completamento del flusso accettato dai componenti, non il recupero degli eventi persi a monte. Le run senza perdite devono quindi essere identificate tramite riconciliazione dei contatori e controlli sui risultati. Negli esperimenti di saturazione o rete degradata le perdite sono a loro volta un risultato da riportare; un throughput apparentemente stabile ottenuto scartando dati non soddisfa il requisito di scalabilità.

Il minimo dei watermark Edge evita risultati prematuri ma può trattenere finestre in memoria quando un Edge non avanza. Un timeout del runner interrompe una prova incompleta senza autorizzare il Global a ignorare la sorgente. Non sono implementati checkpoint, migrazione dello stato o una politica di esclusione dinamica degli Edge.

Il workload bilancia i sensori per zona e riguarda un solo tipo di dispositivo in un solo mese. Le medie sono pesate per numero di campioni validi: sensori più frequenti contribuiscono maggiormente. I risultati non possono essere generalizzati automaticamente ad altri dataset, algoritmi più costosi o installazioni geograficamente distribuite. Inoltre, la condivisione degli host e i limiti per container introducono contesa; la scalabilità delle repliche entro un host non dimostra un'illimitata capacità di espansione.

VII. CONCLUSIONI

L'applicazione realizza una pipeline Edge-Cloud in Go con validazione locale, aggregazione gerarchica e comunicazione persistente nei passaggi Kafka. La separazione fra partizioni logiche ed esecutori mantiene invariato il significato degli output al variare delle repliche. Watermark espliciti, progresso di partizione ed EOS permettono di concludere le finestre soltanto quando ogni contributo è determinato, includendo le sorgenti senza dati.

Le verifiche funzionali e gli artefatti preliminari forniscono una base per la valutazione, ma non consentono ancora di concludere che la scalabilità richiesta sia dimostrata. Il completamento del lavoro richiede una campagna EC2 valida e ripetuta, l'analisi dei tempi di disponibilità dei risultati e la valutazione di latenza e banda emulate. Le conclusioni quantitative dovranno essere aggiornate dopo tali esperimenti, conservando espliciti i limiti dello stato volatile e della telemetria con possibili perdite.

RIFERIMENTI BIBLIOGRAFICI

- [1] V. Cardellini, *Progetto A2: Applicazione Distribuita nel Compute Continuum*, Sistemi Distribuiti e Cloud Computing, Università degli Studi di Roma Tor Vergata, A.A. 2025/26, pp. 1–3.
- [2] Sensor.Community, *Get historical datas*, documentazione della comunità sull'archivio delle osservazioni. <https://forum.sensor.community/t/get-historical-datas/1648>. Consultato l'11 settembre 2026.
- [3] Apache Software Foundation, *Apache Kafka – Design*, documentazione 4.1. <https://kafka.apache.org/41/design/design/>. Consultato l'11 settembre 2026.
- [4] Apache Software Foundation, *Apache Avro 1.12.0 – Specification*. <https://avro.apache.org/docs/1.12.0/specification/>. Consultato l'11 settembre 2026.